

EXPERIMENT 2

Stages of Compilation and Execution of a C Program

1. Aim

To demonstrate the different stages involved in compiling and executing a C program using GCC, and to calculate the area of a circle.

2. Objective

To understand preprocessing, compilation, assembly, linking, and execution of a C program and to observe the intermediate files generated using the GCC -save-temps option.

3. Software / Tools Required

- Linux operating system
- GCC (GNU Compiler Collection)
- Terminal
- Text editor (e.g., nano or VS Code)

4. C Program

```
#include <stdio.h>

int main()
{
    float radius, area;

    printf("Enter radius of circle: ");
    scanf("%f", &radius);

    area = 3.14159 * radius * radius;

    printf("Area of circle = %.2f\n", area);

    return 0;
}
```

5. Procedure

Step 1: Create the source file

Create a file named circle.c and enter the C program given above.

Step 2: Compile the program normally

Compile the program using GCC. This creates the default executable a.out.

Step 3: Execute the program

Run the executable and enter a radius to calculate the area of the circle.

Step 4: Generate and preserve temporary files

Use `gcc -save-temps circle.c`. The `-save-temps` option tells GCC to preserve intermediate compilation files.

Step 5: Observe the preprocessed file

The file `circle.i` contains the preprocessed source code. Header files and preprocessing directives have been processed at this stage.

Step 6: Observe the assembly file

The file `circle.s` contains assembly language generated from the preprocessed C source.

Step 7: Observe the object file

The file `circle.o` is an object file containing machine-level code generated by the assembler.

Step 8: Linking and executable

The linker combines the object code with required libraries and produces the final executable.

Step 9: Execute the final program

Run `./a.out` and verify the calculated area.

6. Important Commands

```
gcc circle.c
./a.out
gcc -save-temps circle.c
ls
head -n 30 circle.i

head -n 30 circle.s
file circle.o
file a.out
./a.out
```

7. Stages of Compilation

Stage	File	Description
Source Code	<code>circle.c</code>	Original C source program written by the programmer.
Preprocessing	<code>circle.i</code>	Processes directives such as <code>#include</code> and produces preprocessed C code.
Compilation	<code>circle.s</code>	Converts the preprocessed C code into assembly language.
Assembly	<code>circle.o</code>	Converts assembly language

		into machine/object code.
Linking	a.out	Combines object code with required libraries to create an executable.
Execution	Output	The executable is run and the area of the circle is displayed.

8. Compilation Flow

```

circle.c
|
v
Preprocessor
|
v
circle.i
|
v
Compiler
|
v
circle.s
|
v
Assembler
|
v
circle.o
|
v
Linker
|
v
a.out
|
v
Execution / Output

```

9. Expected Output

```

Enter radius of circle: 5
Area of circle = 78.54

```

The output value will depend on the radius entered by the user. For a radius of 5 units, the area is approximately 78.54 square units.

10. Screenshots / Evidence

Figure 1. Creation of the Experiment2 directory, source file, and C program.

```
(kali㉿kali)-[~]
$ cd ~/Desktop

(kali㉿kali)-[~/Desktop]
$ mkdir Experiment2

(kali㉿kali)-[~/Desktop]
$ cd Experiment2

(kali㉿kali)-[~/Desktop/Experiment2]
$ vi circle.c

(kali㉿kali)-[~/Desktop/Experiment2]
$ cat circle.c
#include <stdio.h>
int main()
{
    float radius, area;
    printf("Enter radius of circle: ");
    scanf("%f", &radius);
    area = 3.14159 * radius * radius;
    printf("Area of circle = %.2f\n",area);
    return 0;
}
```

Figure 2. Normal compilation, generated executable, and program execution.

```
(kali㉿kali)-[~/Desktop/Experiment2]
$ gcc circle.c

(kali㉿kali)-[~/Desktop/Experiment2]
$ ls
a.out circle.c

(kali㉿kali)-[~/Desktop/Experiment2]
$ ./a.out
Enter radius of circle: 5
Area of circle = 78.54
```

Figure 3. GCC -save-temps command and generated intermediate files.

```
(kali㉿kali)-[~/Desktop/Experiment2]
$ gcc -save-temps circle.c

(kali㉿kali)-[~/Desktop/Experiment2]
$ ls
a-circle.i a-circle.o a-circle.s a.out circle.c
```

Figure 4. Preprocessed source file circle.i.

```
(kali㉿kali)-[~/Desktop/Experiment2]
$ head -n 30 a-circle.i
# 0 "circle.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "circle.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 415 "/usr/include/features.h" 3 4
# 1 "/usr/include/features-time64.h" 1 3 4
# 20 "/usr/include/features-time64.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 21 "/usr/include/features-time64.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 1 3 4
# 19 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 20 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 2 3 4
# 22 "/usr/include/features-time64.h" 2 3 4
# 416 "/usr/include/features.h" 2 3 4
# 523 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 1 3 4
# 730 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 731 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/long-double.h" 1 3 4
# 732 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 524 "/usr/include/features.h" 2 3 4
```

Figure 5. Assembly file circle.s.

```
(kali@kali)-[~/Desktop/Experiment2]
$ head -n 30 a-circle.s
.file "circle.c"
.text
.section      .rodata
.LC0:
.string "Enter radius of circle: "
.LC1:
.string "%f"
.LC3:
.string "Area of circle = %.2f\n"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
subq $16, %rsp
leaq .LC0(%rip), %rax
movq %rax, %rdi
movl $0, %eax
call printf@PLT
leaq -8(%rbp), %rax
leaq .LC1(%rip), %rdx
movq %rax, %rsi
movq %rdx, %rdi
movl $0, %eax
```

Figure 6. Object/executable file information and final program execution.

```
(kali@kali)-[~/Desktop/Experiment2]
$ file a-circle.o
a-circle.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped

(kali@kali)-[~/Desktop/Experiment2]
$ file a.out
a.out: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=f458522d0df7945e5540f4a7f32558922e80d9da, for GNU/Linux 3.2.0, not stripped

(kali@kali)-[~/Desktop/Experiment2]
$ ./a.out
Enter radius of circle: 5
Area of circle = 78.54
```

11. Result

The C program for calculating the area of a circle was successfully compiled and executed. The stages of compilation—preprocessing, compilation, assembly, and linking—were studied, and the intermediate files generated using the `-save-temps` option were observed.

12. Conclusion

The experiment demonstrates that a C program passes through several stages before it becomes an executable program. GCC can preserve the intermediate files using `-save-temps`, making it possible to study the transformation from C source code to assembly, object code, and finally an executable program.